

Roadmap URI Pack v1 — SOUP v2

Fase 2, compatible y **aditiva** al *Roadmap del Sistema de URIs (Core)*. No rompe contratos existentes ni modifica `entity_uri_registry`; sólo **lee** de él y añade nuevas capacidades para adopción y valor inmediato.

Índice

- [0\) Contexto y objetivos](#)
 - [1\) Alcance y no-objetivos](#)
 - [2\) Reglas de compatibilidad](#)
 - [3\) Prompts para agente de IA](#)
 - [4\) Plantillas de código \(stubs\)](#)
 - [5\) Arquitectura \(alto nivel\)](#)
 - [6\) Checks y tests](#)
 - [7\) Recomendaciones de implementación](#)
 - [8\) Tutorial para el humano \(paso a paso\)](#)
 - [9\) Criterios de aceptación y KPIs](#)
 - [A\) Apéndice: ejemplos \(JSON-LD, oEmbed, embed HTML\)](#)
-

0) Contexto y objetivos

Objetivo: Transformar los URIs del Core en **valor visible** para artistas, sellos, venues y prensa, con: shortlinks/QR, embeds, SEO (JSON-LD/oEmbed), importadores, verificación (claim) y analytics mínimas.

Usuarios: artistas/equipos (self-serve), medios y venues (embeds), equipo interno (analytics).

1) Alcance y no-objetivos

Incluye (v1):

- Shortlinks (`/s/{code}`) y generación de QR PNG.
- Landing pública mínima (usa vistas actuales) con JSON-LD + OpenGraph.
- Endpoint `GET /oembed` y **embeds** (`/embed/{gid}/upcoming`, `/embed/{gid}/listen`).
- Importador **Bandcamp** (prefill de portada, links y tracklist).
- **Claim** por token verificable.
- **Analytics** mínimas (view/click) con agregaciones diarias.

No incluye (v1): pagos, dashboards complejos, importadores Spotify/IG/Ticketing (dejados para v1.1+), internacionalización avanzada.

2) Reglas de compatibilidad

1. No se modifican ni se rompen contratos del Core; sólo se **agregan** endpoints y campos renderizados.
2. Las páginas y listados existentes siguen funcionando; se añade JSON-LD/OG sin cambiar estructuras.
3. Embeds y shortlinks funcionan aunque la banda/álbum no tenga datos enriquecidos (estados vacíos amables).

3) Prompts para agente de IA

A) Shortlinks & QR

Crea la tabla `shortlinks(code TEXT pk, target\_uri TEXT NOT NULL, gid TEXT NOT NULL, owner\_id UUID, created\_at TIMESTAMPTZ DEFAULT NOW())`.

Endpoints:

- POST /tools/shortlinks {gid|target_uri} → {code, url, qr_png_url}
- GET /s/{code} → 302 a target_uri
- GET /tools/shortlinks/{code}/qr.png → image/png

Agrega helper para generar PNG QR desde una URL.

B) oEmbed & JSON-LD

Agrega `GET /oembed?url=...` que devuelva JSON con {version:"1.0", type:"rich", provider_name:"SOUP", html:"<iframe ...>", width, height}. Inserta JSON-LD (schema.org MusicGroup/MusicAlbum) en las vistas públicas de banda/álbum usando datos del Core.

C) Embeds

Crea `GET /embed/{gid}/upcoming` (lista simple de próximas fechas) y `GET /embed/{gid}/listen` (botones de plataformas desde /links).

Sirve HTML minimalista, sin JS externo. Asegura cabeceras para permitir <iframe> (CSP frame-ancestors).

D) Importador Bandcamp (mínimo viable)

Crea POST /tools/import/bandcamp {url} → {title, cover_url, links:[...], tracklist:[...]}

Implementa parser básico; si falla, devuelve estructura con campos vacíos para pegar manualmente.

E) Claim / verificación

Crea tabla `entity\_claims(entity\_gid TEXT, verify\_token TEXT, status TEXT, requested\_by UUID, verified\_at TIMESTAMPTZ)`.

Endpoints:

- POST /claim/request {entity_gid} → genera verify_token y devuelve instrucciones.
- POST /claim/verify {entity_gid, proof_url} → verifica presencia del token (consulta GET a proof_url o matching rel=me) y marca status=verified.

F) Analytics mínimas

Crea tabla `analytics\_events(id bigserial pk, entity\_gid TEXT, type TEXT, subresource TEXT, referrer TEXT, ua\_hash TEXT, created\_at TIMESTAMPTZ DEFAULT NOW())`.

Endpoints:

- POST /public/track {entity_gid, type:view|click, subresource, referrer}
 - GET /admin/analytics/summary?gid=...&from=...&to=... → vistas/clics por día
- Incluye throttling básico (IP+ua_hash) y minimiza PII (hash de UA).

4) Plantillas de código (stubs)

Nota: nombres y ubicaciones pensados para FastAPI + SQLAlchemy en `backend/app`.
Ajustar a tu árbol.

4.1 Modelos SQLAlchemy (nuevos)

```
# app/models_shortlinks.py
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, func
from app.database import Base

class Shortlink(Base):
    __tablename__ = "shortlinks"
    code: Mapped[str] = mapped_column(String, primary_key=True)
    target_uri: Mapped[str] = mapped_column(String, nullable=False)
    gid: Mapped[str] = mapped_column(String, nullable=False, index=True)
    owner_id: Mapped[str | None] = mapped_column(String, nullable=True)
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
```

```
# app/models_analytics.py
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, func, BigInteger
from app.database import Base
```

```

class AnalyticsEvent(Base):
    __tablename__ = "analytics_events"
    id: Mapped[int] = mapped_column(BigInteger, primary_key=True,
autoincrement=True)
    entity_gid: Mapped[str] = mapped_column(String, index=True)
    type: Mapped[str] = mapped_column(String) # view|click
    subresource: Mapped[str | None] = mapped_column(String)
    referrer: Mapped[str | None] = mapped_column(String)
    ua_hash: Mapped[str | None] = mapped_column(String)
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

```

```

# app/models_claims.py
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, func
from app.database import Base

class EntityClaim(Base):
    __tablename__ = "entity_claims"
    entity_gid: Mapped[str] = mapped_column(String, primary_key=True)
    verify_token: Mapped[str] = mapped_column(String, nullable=False)
    status: Mapped[str] = mapped_column(String, default="pending")
    requested_by: Mapped[str | None] = mapped_column(String)
    verified_at: Mapped[DateTime | None] =
mapped_column(DateTime(timezone=True))
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

```

4.2 Schemas Pydantic (nuevos)

```

# app/schemas_pack.py
from pydantic import BaseModel, HttpUrl
from typing import Optional, List

class ShortlinkCreate(BaseModel):
    gid: str
    target_uri: HttpUrl

class ShortlinkOut(BaseModel):
    code: str
    url: str
    qr_png_url: str

class OEmbedResponse(BaseModel):
    version: str = "1.0"
    type: str = "rich"
    provider_name: str = "SOUP"
    html: str

```

```

width: int = 600
height: int = 400

class AnalyticsEventIn(BaseModel):
    entity_gid: str
    type: str
    subresource: Optional[str] = None
    referrer: Optional[str] = None

class ClaimRequestIn(BaseModel):
    entity_gid: str

class ClaimVerifyIn(BaseModel):
    entity_gid: str
    proof_url: HttpUrl

class ImportBandcampIn(BaseModel):
    url: HttpUrl

class ImportBandcampOut(BaseModel):
    title: Optional[str]
    cover_url: Optional[HttpUrl]
    links: List[str] = []
    tracklist: List[str] = []

```

4.3 Utils

```

# app/utils/qr.py
import io
try:
    import qrcode
except ImportError:
    qrcode = None

def qr_png_bytes(url: str) -> bytes:
    if not qrcode:
        raise RuntimeError("Instala 'qrcode' para generar PNGs")
    img = qrcode.make(url)
    buf = io.BytesIO()
    img.save(buf, format="PNG")
    return buf.getvalue()

```

```

# app/utils/ld.py (JSON-LD builders)
from typing import Dict

def band_ld(band: dict) -> Dict:
    return {
        "@context": "https://schema.org",
        "@type": "MusicGroup",

```

```

        "name": band.get("name"),
        "url": band.get("uri"),
        "sameAs": [l for l in band.get("links", [])],
    }

def album_ld(album: dict) -> Dict:
    return {
        "@context": "https://schema.org",
        "@type": "MusicAlbum",
        "name": album.get("name"),
        "byArtist": {"@type": "MusicGroup", "name":
album.get("artist_name")},
        "url": album.get("uri"),
    }

```

```

# app/utils/ua.py
import hashlib

def ua_fingerprint(ua: str, ip: str | None = None) -> str:
    s = (ua or "") + "|" + (ip or "")
    return hashlib.blake2b(s.encode(), digest_size=8).hexdigest()

```

4.4 Routers (stubs)

```

# app/routers/shortlinks.py
from fastapi import APIRouter, Response
from app.schemas_pack import ShortlinkCreate, ShortlinkOut
from app.utils.qr import qr_png_bytes

router = APIRouter(prefix="", tags=["Shortlinks"])

@router.post("/tools/shortlinks", response_model=ShortlinkOut)
def create_shortlink(payload: ShortlinkCreate):
    # 1) generar code corto (base36/blake2b), 2) persistir, 3) devolver URLs
    ...

@router.get("/s/{code}")
def resolve_shortlink(code: str):
    # 302 a target_uri
    ...

@router.get("/tools/shortlinks/{code}/qr.png")
def qr_png(code: str):
    # genera PNG en tiempo real
    png = qr_png_bytes(f"https://soup.market/s/{code}")
    return Response(content=png, media_type="image/png")

```

```

# app/routers/oembed.py
from fastapi import APIRouter, HTTPException
from app.schemas_pack import OEmbedResponse

router = APIRouter(prefix="", tags=["oEmbed"])

@router.get("/oembed", response_model=OEmbedResponse)
def oembed(url: str):
    # validar URL de SOUP y devolver iframe HTML
    html = f'<iframe src="{url}" width="600" height="400" frameborder="0"
loading="lazy"></iframe>'
    return OEmbedResponse(html=html)

```

```

# app/routers/embeds.py
from fastapi import APIRouter, Response

router = APIRouter(prefix="/embed", tags=["Embeds"])

@router.get("/{gid}/upcoming")
def upcoming(gid: str):
    # HTML mínimo con lista de fechas (puede ser estático en v1)
    html = "<div class='soup-embed-upcoming'>No hay fechas cargadas aún</div>"
    return Response(content=html, media_type="text/html")

@router.get("/{gid}/listen")
def listen(gid: str):
    # HTML mínimo con botones a plataformas (desde /links)
    html = "<div class='soup-embed-listen'><a href='#'>Spotify</a> · <a href='#'>Bandcamp</a></div>"
    return Response(content=html, media_type="text/html")

```

```

# app/routers/importers.py
from fastapi import APIRouter
from app.schemas_pack import ImportBandcampIn, ImportBandcampOut

router = APIRouter(prefix="/tools/import", tags=["Importers"])

@router.post("/bandcamp", response_model=ImportBandcampOut)
def import_bandcamp(payload: ImportBandcampIn):
    # Parser muy básico o placeholder (v1): devuelve estructura vacía si falla
    return ImportBandcampOut(title=None, cover_url=None, links=[payload.url], tracklist=[])

```

```

# app/routers/claims.py
from fastapi import APIRouter

```

```

from app.schemas_pack import ClaimRequestIn, ClaimVerifyIn

router = APIRouter(prefix="", tags=["Claims"])

@router.post("/claim/request")
def claim_request(payload: ClaimRequestIn):
    # generar verify_token y guardar pending
    ...

@router.post("/claim/verify")
def claim_verify(payload: ClaimVerifyIn):
    # chequear proof_url contiene token; si OK → status=verified
    ...

```

```

# app/routers/analytics.py
from fastapi import APIRouter, Request
from app.schemas_pack import AnalyticsEventIn
from app.utils.ua import ua_fingerprint

router = APIRouter(prefix="", tags=["Analytics"])

@router.post("/public/track")
async def track(evt: AnalyticsEventIn, request: Request):
    # throttling simple por fingerprint y ventana corta; persistir evento
    ...

@router.get("/admin/analytics/summary")
def summary(gid: str, date_from: str, date_to: str):
    # agregar por día (COUNT) view/click
    ...

```

5) Arquitectura (alto nivel)

```

[FastAPI]
├─ /s/{code}          (302 shortlink)
├─ /oembed            (oEmbed JSON)
├─ /embed/{gid}/upcoming (HTML embed)
├─ /embed/{gid}/listen  (HTML embed)
├─ /public/track       (collector analytics)
├─ /tools/shortlinks   (crear + QR)
├─ /tools/import/bandcamp (prefill)
└─ /claim/*           (verificación)

[DB]
├─ entity_uri_registry (Core, sólo lectura)
└─ shortlinks         (nuevo)

```


└─ analytics_events	(nuevo)
└─ entity_claims	(nuevo)

6) Checks y tests

Unit

- `qr_png_bytes()` genera PNG válido.
- `band_ld()/album_ld()` conformes a schema básico.
- `ua_fingerprint()` determinista.

Integración (pytest)

- `POST /tools/shortlinks` crea código único → `GET /s/{code}` hace 302.
- `GET /oembed` devuelve estructura válida.
- `GET /embed/*` retorna `text/html` y es embebible (cabeceras correctas).
- Claim: `request` → token creado; `verify` con `proof_url` válida marca `verified`.
- Analytics: `POST /public/track` persiste evento y `summary` agrega por día.

E2E

- Página externa carga `<iframe src="/embed/{gid}/listen">` y muestra botones.
- QR impreso abre la landing correcta en móvil.

No-regresión

- Listados y páginas públicas del Core siguen funcionando idénticas.

7) Recomendaciones de implementación

- **Seguridad/CSP:** usa `Content-Security-Policy: frame-ancestors 'self' *dominios_permitidos*` para embeds; evita `X-Frame-Options: DENY` en rutas `/embed/*`.
- **Privacidad:** en analytics no guardes IP cruda; sólo hash UA+IP.
- **Rate limiting:** ventana deslizante para `/public/track`.
- **Migrations:** tablas nuevas en migraciones separadas, reversibles.
- **Git:** rama `feature/uri-pack-v1`; PRs pequeños (shortlinks, embeds/oEmbed, importador, claim, analytics).
- **IA workflow:** Continue para stubs, aider para cambios multi-archivo, Code Assist/Windsurf para transformaciones grandes.

8) Tutorial para el humano (paso a paso)

1. **Crear rama** `feature/uri-pack-v1`.
2. **Dependencias:** `pip install qrcode` (opcional para PNGs locales).
3. **Modelos y migraciones:** crear `shortlinks`, `analytics_events`, `entity_claims`.

4. **Routers:** añadir y montar en `main.py` (`shortlinks`, `oembed`, `embeds`, `importers`, `claims`, `analytics`).
5. **JSON-LD/OG:** insertar script JSON-LD en vistas públicas de band/album.
6. **Probar rápido** (`curl`):

```
# crear shortlink
POST /tools/shortlinks {gid, target_uri}
# redirigir
GET /s/{code}
# oEmbed
GET /oembed?url=https://soup.market/band/slug~short
# track
POST /public/track {entity_gid, type:"view"}
```

1. **QA de embeds:** abrir los iframes en una página HTML externa.
2. **Pilot:** habilitar para 5 bandas; imprimir QR; medir clics 1 semana.

9) Criterios de aceptación y KPIs

Aceptación

- Shortlink (302) + QR operativo.
- oEmbed válido y embeds visibles.
- JSON-LD válido en Rich Results.
- Claim operativo y estado `verified` visible en ficha.
- Analytics con agregados diarios.

KPIs

- % bandas con URI reclamado.

de shortlinks/QR generados. - CTR a streaming/tickets desde landing/embeds. - % páginas con JSON-LD válido.

A) Apéndice: ejemplos (JSON-LD, oEmbed, embed HTML)

A.1 JSON-LD (banda)

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "MusicGroup",
  "name": "Banda Ñam",
  "url": "https://soup.market/band/banda-nam-x7k3",
  "sameAs": [
    "https://bandcamp.com/bandanam",
```

```
    "https://instagram.com/bandanam"
  ]
}
</script>
```

A.2 oEmbed (respuesta)

```
{
  "version": "1.0",
  "type": "rich",
  "provider_name": "SOUP",
  "html": "<iframe src=\"https://soup.market/embed/01J8A.../listen\"
width=\"600\" height=\"120\"></iframe>",
  "width": 600,
  "height": 120
}
```

A.3 HTML de embed mínimo (``)

```
<div class="soup-embed-listen" style="font:14px system-ui;display:flex;gap:
8px">
  <a href="https://open.spotify.com/..." target="_blank"
rel="noopener">Spotify</a>
  <a href="https://bandcamp.com/..." target="_blank"
rel="noopener">Bandcamp</a>
  <a href="https://music.youtube.com/..." target="_blank" rel="noopener">YouTube
Music</a>
</div>
```

Resumen: Este *URI Pack v1* convierte el Core en producto: links cortos y QR, embeds que cualquiera puede pegar, SEO listo para buscadores, importador rápido desde Bandcamp, verificación para confianza y métricas básicas para demostrar impacto. Todo **sin romper** el roadmap anterior.